01 Introduction to KNN

Definition and Basics

- Instance learning algorithm: KNN is an instance based supervised learning algorithm that belongs to the category of lazy learning. Its core idea is to calculate the distance between the sample to be classified and all samples in the training set, and select the K nearest neighbors for category voting decision-making. The algorithm does not construct an explicit model, but directly stores the entire training set for prediction.

- Distance measurement methods: Common distance measurements include Euclidean Distance, Manhattan Distance, and Minkowski Distance. The Euclidean distance is applicable to continuous feature spaces; Manhattan distance is more robust to outliers and suitable for high-dimensional sparse data.

- The selection of parameter K: The K value determines the smoothness of the decision boundary. A smaller K value (such as K=1) can cause the model to be sensitive to noise and overfit, while a larger K value can make the decision boundary too smooth and potentially underfit. Usually, the optimal K value is selected through cross validation, with an empirical range of odd numbers between 3-10 to avoid tie ticket situations.

Key Characteristics

- Non parametric feature: KNN does not require any assumptions about data distribution (such as linear separability) and can naturally adapt to complex decision boundaries. But this also leads to a lack of interpretability in the algorithm, which cannot provide parameter information such as feature importance.

- High computational complexity: In the prediction phase, it is necessary to calculate the distance between the test samples and all training samples, with a time complexity of O (nd) (n is the number of samples, d is the feature dimension). The solution includes using KD Tree or Ball Tree data structures, which can reduce complexity to O (log n).

- Feature scaling sensitivity: Due to the dependence on distance calculation, the algorithm is sensitive to feature scale.

- Standardization processing : must be performed (such as Z-score normalization or Min Max normalization), otherwise large-scale features will dominate distance calculation.

- Category imbalance impact: When a certain category has a large proportion of samples, simple voting can lead to predictions biased towards the majority class. Improvement methods include distance weighted voting (giving higher weights to neighbors) or using SMOTE oversampling techniques to balance the dataset.

KNN with spatial data structures

- KD-Trees and Ball Trees—core spatial data structures used to optimize nearest-neighbor searches in KNN algorithms.

- Choose KD Tree for low-dimensional efficiency; Ball Tree for high-dimensional resilience. Hybrid approaches (e.g., VP-Tree) exist but are less common in practice.

KD-Tree (k-Dimensional Tree)

- **Core Structure**
  - A binary tree for organizing points in k-dimensional space.
  - Construction: Recursively splits data by selecting a dimension (e.g., highest variance) and splitting at the median point.
  - Space Partitioning: Uses axis-aligned hyperplanes to divide space into hyperrectangles.
- **Pros & Cons**
  - ☑Low-dimensional efficiency: Fast construction and search for d ≤ 10 dimensions.
  - ☑Optimized search: Backtracking and pruning reduce distance calculations.
  - ❌Curse of dimensionality: Performance degrades sharply in high dimensions due to overlapping regions.
  - ❌Rigid partitioning: Axis-aligned splits may poorly fit data geometry.
- **Applications**
  - Nearest-neighbor search in 2D/3D (e.g., GIS, physics simulations).
  - Python: scipy.spatial.KDTree.

Ball Tree

- **Core Structure**
  - Designed to address KD Tree's high-dimensional limitations.
  - Construction: Recursively partitions data into nested hyperspheres (each node = centroid + radius).
- **Pros & Cons**
  - ☑High-dimensional robustness: Hyperspheres adapt better to data geometry than hyperrectangles.
  - ☑Triangular inequality optimization: Excludes distant balls during search, reducing computations.
  - ❌Higher build cost: Calculating tight hyperspheres is more complex than median splits.
  - ❌Rebalancing needs: May require periodic reconstruction for dynamic data.
- **Applications**
  - High-dimensional similarity search (e.g., NLP embeddings, image features).
  - Python: sklearn.neighbors.BallTree.

Key Differences： KD-Tree vs Ball Tree

Criterion	KD-Tree	Ball Tree
Space Partitioning	Axis-aligned hyperplanes → hyper-rectangles	Hyperspheres
Optimal Dimensionality	$d \leqslant 10–20$	$d > 20$
Build Complexity	$O(n \log n)$	$O(n^2)$ (worst-case)
Search Complexity	$O(\log n)$ (low d)	$O(\log n)$ (high d)
Data Distribution	Best for axis-aligned, uniform data	Tolerates clustered/irregular data

Common Use Cases

- Medical diagnosis prediction: it is used for disease risk prediction in the medical field. For example, early diagnosis of diabetes can analyze the similarity between clinical indicators (blood sugar, BMI, etc.) and historical cases. Research shows that the average accuracy rate of KNN in breast cancer detection can reach more than 85% (attention should be paid to adjusting the imbalance of kappa coefficient processing categories when implementing R language).

- Recommendation system construction: Used in e-commerce for collaborative filtering recommendations, based on user historical behavior to find similar user groups (K-nearest neighbors) and recommend their preferred products. In practical applications, cosine similarity is often used instead of traditional distance measurement to improve performance.

- Image classification task: As a baseline method for computer vision, it is used for handwritten digit recognition (such as the MNIST dataset) or simple object classification. When combined with SIFT/HOG feature extraction, traditional KNN still has competitiveness on small datasets, but attention should be paid to the curse of dimensionality.

02 How KNN Works

Distance Metrics (Euclidean, Manhattan)

01

Euclidean Distance

calculates the straight-line distance between two points in a multidimensional space, with the formula $\sqrt{(\Sigma (x_i-y_i)^2)}$. It is suitable for scenes with continuous features and equal importance in each dimension, such as **pixel difference analysis in image recognition**.

02

Manhattan Distance

Measuring distance through the **sum of absolute coordinate differences** ($\Sigma |x_i-y_i|$), insensitive to outliers, commonly **used in high-dimensional sparse data (such as text classification) or grid path planning problems.**

03

Minkowski Distance

Generalization of Euclidean and Manhattan distances, with the formula $(\Sigma |x_i-y_i|^p)^{1/p}$. When p=1, it is the Manhattan distance, and when p=2, it is the Euclidean distance. It can **flexibly adapt to different data distribution needs**.

KNN Core Formula: Euclidean Distance

KNN measures similarity using **Euclidean distance** between a query point q and a data point p:

$$d(p,q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

p, q: n-dimensional vectors (e.g., features of a data point).

p_i, q_i: Values of the i-th feature.

2. Prediction Rules

Classification: Predict the **mode** (most frequent class) of the k nearest neighbors.

Regression: Predict the **average** of the k nearest neighbors' target values.

Example 1: Classification (Iris Species)

Goal: Predict the species of a new flower with features $[5.0, 3.3, 1.4, 0.2]$.

Step 1: Calculate Distance to a training sample $[4.6, 3.4, 1.4, 0.3]$:

$$d = \sqrt{(5.0 - 4.6)^2 + (3.3 - 3.4)^2 + (1.4 - 1.4)^2 + (0.2 - 0.3)^2} = \sqrt{0.16 + 0.01 + 0 + 0.01} \approx 0.424$$

Step 2: Find $k = 3$ Nearest Neighbors

Assume their species are: ["setosa", "versicolor", "setosa"].

Step 3: Predict → "setosa" (majority vote).

Example 2: Regression (Income Prediction)

Goal: Predict income for a 47-year-old with 2 years of experience.

Training Data:

Age	Experience	Income
45	1	40,000
50	3	41,000
48	2	41,500

Step 1: Calculate Distance to $(45, 1)$:

$$d = \sqrt{(47-45)^2 + (2-1)^2} = \sqrt{4+1} \approx 2.236$$

Step 2: Find $k = 3$ **Nearest Neighbors** (all 3 points).

Step 3: Predict Income → Average:

$$\frac{40{,}000 + 41{,}000 + 41{,}500}{3} = 40{,}833.33$$

Manhattan Distance (L1 Distance)

Formula

The Manhattan distance (or *taxicab distance*) between two points $x = (x_1, x_2, \ldots, x_n)$ and $y = (y_1, y_2, \ldots, y_n)$ in n-dimensional space is the sum of absolute differences along each dimension:

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

This is a special case of the Minkowski distance when $p = 1$.

Numeric Example

Consider two 2D points:

Point A: $(1, 3)$

Point B: $(4, 7)$

Calculation:

$$d = |1 - 4| + |3 - 7| = |-3| + |-4| = 3 + 4 = 7$$

Interpretation: The distance is 7 units, mimicking a path along grid lines (e.g., moving 3 units horizontally and 4 units vertically).

Minkowski Distance

Formula

The Minkowski distance generalizes Euclidean ($p = 2$) and Manhattan ($p = 1$) distances. For points x and y, it is defined as:

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p\right)^{\frac{1}{p}}$$

Key cases:

$p = 1$: Manhattan distance.

$p = 2$: Euclidean distance.

$p\infty$: Chebyshev distance (maximum dimension difference).

Numeric Example

Using the same points $A(1,3)$ and $B(4,7)$, compute for $p = 3$:

1.Compute absolute differences raised to $p = 3$:

$$|1 - 4|^3 = 3^3 = 27, |3 - 7|^3 = 4^3 = 64$$

2.Sum and take the p-th root:

$$d = (27 + 64)^{\frac{1}{3}} = 91^{\frac{1}{3}} \approx 4.50$$

Comparison:

$p = 1$: Distance = 7 (Manhattan).

$p = 2$: Distance = $\sqrt{3^2 + 4^2} = 5$ (Euclidean).

$p = 3$: Distance ≈ 4.50.

Distance Metric in KNN

Parameter tuning: In scikit-learn, use metric="minkowski" and set p to switch between distance metrics (e.g., p=1 for Manhattan, p=2 for Euclidean).

When to use which:

Manhattan ($p = 1$): Robust for high-dimensional data or outliers; scales better than Euclidean distance when features have varying units.

Minkowski ($p \geq 3$): Emphasizes larger dimension differences (e.g., $/x_i - y_i/^p$ amplifies disparities). Optimal p is data-dependent—use cross-validation to select.

Best practice: Always normalize features so no single dimension dominates distance calculations.

Choosing the Value of K

- Too small K leads to overfitting: If K=1, the model is sensitive to noise, the decision boundary is complex, and it is susceptible to local outlier interference, reducing generalization ability (such as the appearance of "jagged edges" in the classification boundary).

- Excessive K leads to underfitting: When K approaches the total number of samples, the model ignores local features and tends to be dominated by the majority class, which may mask the true data distribution (such as misclassification due to overly smooth boundaries).

- Odd value K to avoid tie votes: especially in binary classification, odd value K (such as 3/5) is recommended to prevent situations where the nearest neighbor votes equally and cannot make a decision.

- Cross validation optimization of K-value: By combining GridSearchCV with k-fold cross validation, select the K-value that maximizes the accuracy of the validation set, balancing bias and variance.

GridSearchCV for KNN Optimization

- Core Purpose
- Automates hyperparameter tuning by exhaustively testing predefined combinations. Uses cross-validation to prevent overfitting and evaluate generalization performance.

- Key KNN Hyperparameters
  - n_neighbors: Number of neighbors (e.g., [3,5,7,11] or range(1,50))
  - weights: 'uniform' (equal weighting) or 'distance' (inverse-distance weighting)
  - p: Distance metric (1=Manhattan, 2=Euclidean)
  - leaf_size: Affects tree-based algorithm efficiency (e.g., [10,30,50])

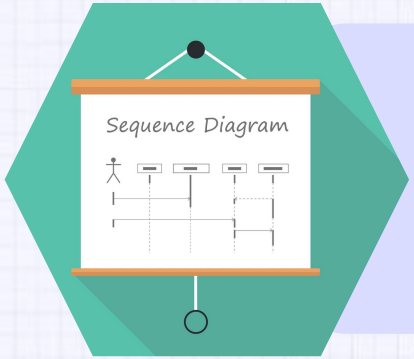
Implementation GridSearchCV for KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
# Define parameter grid
param_grid = {
    'n_neighbors': [3,5,7,11],
    'weights': ['uniform','distance'],
    'p': [1,2]  # Required when weights='distance'
}
# Initialize GridSearch
grid_search = GridSearchCV(
    estimator=KNeighborsClassifier(),
    param_grid=param_grid,
    cv=5,            # 5-fold cross-validation
    scoring='accuracy',
    n_jobs=-1        # Parallel processing
)
# Execute search
grid_search.fit(X_train, y_train)
# Retrieve results
best_model = grid_search.best_estimator_
best_params = grid_search.best_params_  # e.g., {'n_neighbors':5, 'weights':'distance'}
```

Critical Considerations

- Parameter Dependencies: p only affects results when weights='distance'
- Performance Gains: Documented accuracy improvements from 77.8% → 92.6% in medical diagnostics
- Efficiency: Use n_jobs=-1 for multi-core parallelization
- Pipelines: For preprocessing steps, use double underscores in param names (e.g., pca__n_components)

Voting Mechanism for Classification

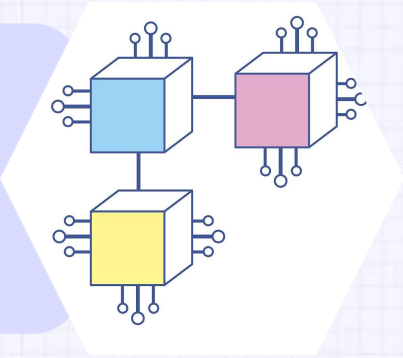


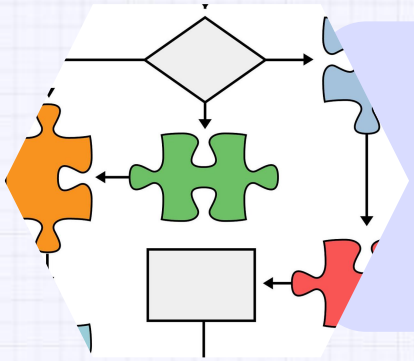
Majority Voting

Counting the most class labels among K nearest neighbors as the prediction result, which is simple and efficient but ignores distance weights. It is suitable for datasets with balanced class distributions.

Distance Weighted Voting

Weights are assigned based on the distance between the nearest neighbor and the target point (e.g. 1/d). Close range samples have a greater impact and can improve robustness in noisy data.





Kernel smoothing

Introducing Gaussian kernel and other functions to weight distance nonlinearly, further refining the contribution of neighbors, suitable for complex boundary scenes (such as fuzzy classification in medical diagnosis).

Majority Voting

Formula:

The predicted class $\hat{y}$ is the class with the highest frequency among the K nearest neighbors:

$$\hat{y} = \text{argmax}_{c \in C} \sum_{i=1}^{K} \mathbb{I}(y_i = c)$$

where C is the set of classes, y_i is the class label of the i-th nearest neighbor, and $\mathbb{I}$ is the indicator function (1 if true, 0 otherwise).

Numeric Example:

Suppose $K = 5$ nearest neighbors have labels: $ClassA, ClassA, ClassB, ClassA, ClassB$.

Count for Class A: 3

Count for Class B: 2

The predicted class is **Class A** (highest count).

Distance Weighted Voting

Formula:

Weights are assigned inversely proportional to distance (e.g., $1/d$). The predicted class is the class with the highest weighted sum:

$$\hat{y} = \text{argmax}_{c \in C} \sum_{i=1}^{K} \frac{\mathbb{I}(y_i = c)}{d_i}$$

where d_i is the distance from the target point to the i-th nearest neighbor.

Numeric Example:

Suppose $K = 3$ nearest neighbors with distances and labels:

Neighbor 1: $d_1 = 1.0$, Class A

Neighbor 2: $d_2 = 2.0$, Class B

Neighbor 3: $d_3 = 0.5$, Class A

Calculate weighted sums:

Class A: $\frac{1}{1.0} + \frac{1}{0.5} = 1 + 2 = 3$

Class B: $\frac{1}{2.0} = 0.5$

The predicted class is **Class A** (highest weighted sum).

Kernel Smoothing

Formula:

A Gaussian kernel (or other function) weights distances nonlinearly. The predicted class is the class with the highest kernel-weighted sum:

$$\hat{y} = \text{argmax}_{c \in C} \sum_{i=1}^{K} \mathbb{I}(y_i = c) \cdot \exp\left(-\frac{d_i^2}{2\sigma^2}\right)$$

where σ is the kernel bandwidth (controls smoothness).

Numeric Example:

Suppose $K = 3$ nearest neighbors with distances and labels, and $\sigma = 1.0$:

Neighbor 1: $d_1 = 1.0$, Class A

Neighbor 2: $d_2 = 2.0$, Class B

Neighbor 3: $d_3 = 0.5$, Class A

Calculate kernel weights (Gaussian):

Neighbor 1: $\exp\left(-\frac{1.0^2}{2 \cdot 1.0^2}\right) = \exp(-0.5) \approx 0.6065$

Neighbor 2: $\exp\left(-\frac{2.0^2}{2 \cdot 1.0^2}\right) = \exp(-2) \approx 0.1353$

Neighbor 3: $\exp\left(-\frac{0.5^2}{2 \cdot 1.0^2}\right) = \exp(-0.125) \approx 0.8825$

Calculate kernel-weighted sums:

Class A: $0.6065 + 0.8825 = 1.489$

Class B: 0.1353

The predicted class is **Class A** (highest kernel-weighted sum).

03 Advantages of KNN

Simple Implementation

Intuitive and easy to understand

The KNN algorithm is based on simple distance metrics such as Euclidean distance and Manhattan distance for classification decisions. Its core idea of "birds of a feather flock together" is very intuitive and can be understood without complex mathematical deductions.

No need for model parameters

Unlike algorithms that require learning weight parameters such as neural networks, KNN only relies on raw data and distance functions, eliminating the complexity of parameter tuning and suitable for beginners to quickly implement prototypes.

Easy code implementation

When using Python libraries such as scikit learn, only a few lines of code are needed to complete data loading, distance calculation, and classification prediction. The underlying logic can also be easily replicated using basic programming languages such as loops and sorting.

No Training Phase

- Lazy learning mechanism: KNN belongs to "Instance Based Learning", which directly stores all training data and performs calculations only during the prediction stage, avoiding the time-consuming model training process of traditional algorithms.

- Real time update capability: Newly added data can be directly incorporated into the training set without the need to retrain the model, making it particularly suitable for data streams or dynamically changing scenarios (such as real-time recommendation systems).

- Retaining original information: Without feature abstraction or dimensionality reduction, the detailed information of the original data (such as local feature distribution) can be fully preserved, which may improve the classification accuracy of complex boundary data.

- Postposition of computing resources: Transfer computing overhead to the prediction stage rather than the training stage, suitable for application scenarios with limited training resources but low prediction response requirements.

Adaptability to New Data

1 Dynamic response to changes in distribution

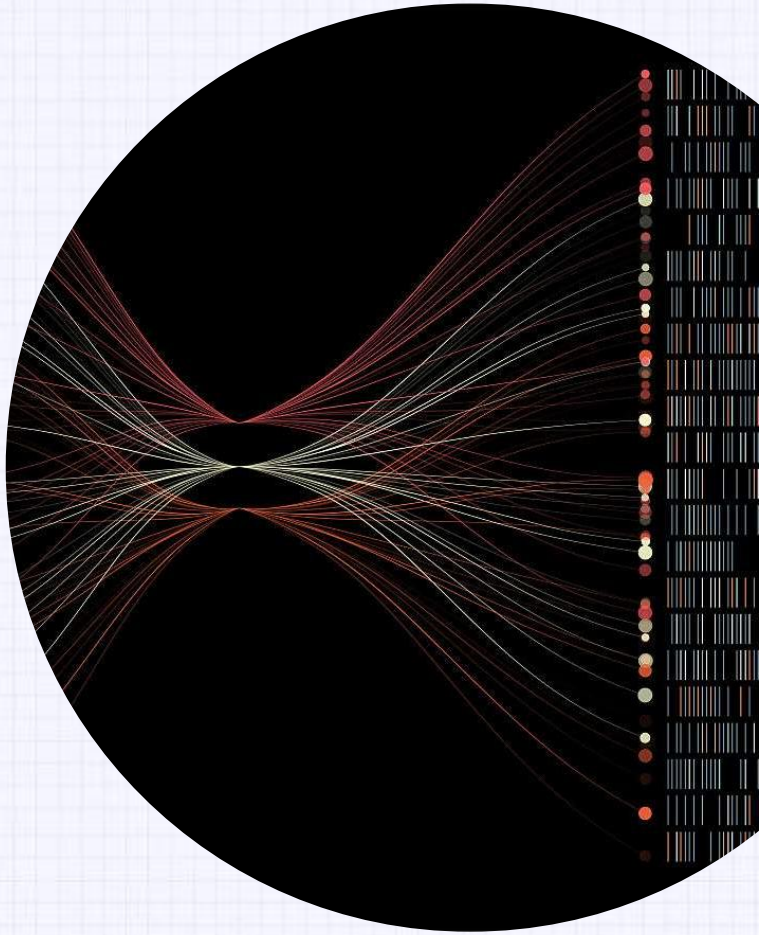
When new data is introduced causing changes in category boundaries, KNN can automatically adapt to the new distribution by recalculating its neighbors, while parameterized models (such as SVM) need to be retrained.

2 Multi class seamless support

Naturally supports multi class problems, without the need for algorithms such as logistic regression that require a "one to many" strategy extension. Only the majority of categories in K neighbors need to be counted.

3 Compatibility with mixed data types

By designing customized distance metrics (such as Hamming distance for categorical variables), it is possible to flexibly handle mixed datasets with numerical, categorical, and even textual features.



04 Limitations of KNN

Computational Cost

01

High memory requirement

The KNN algorithm needs to store all training data in order to calculate the distance between new samples and each training sample during prediction. For large-scale datasets, this consumes a significant amount of memory resources, resulting in low storage and computational efficiency.

02

Slow prediction speed

Due to the need for KNN to calculate the distance between new samples and all training samples during the prediction phase, the prediction time increases linearly with the increase of data volume, making it difficult to meet the high real-time requirements of application scenarios.

03

Not suitable for high-dimensional data

In high-dimensional space, distance calculation becomes complex and the computational workload increases dramatically ("curse of dimensionality"), leading to further decline in algorithm efficiency and even possible failure due to data sparsity.

Sensitivity to Irrelevant Features

- Distance measurement distortion: If the dataset contains a large number of irrelevant features, these features can interfere with distance calculations (such as Euclidean distance), leading to inaccurate similarity judgments and thus reducing the accuracy of model classification or regression.

- Feature scaling dependency: KNN is sensitive to the scale of features. If the numerical range of certain features is much larger than other features (such as age and income), unnormalized data will bias the distance calculation towards large-scale features, affecting the effectiveness of the model.

- The impact of noisy data: Unrelated features or noisy data may mask the true pattern, causing the selection of nearest neighbors to deviate from the actual pattern, especially when the feature dimension is high, this problem is more significant.

- Feature selection is required: To improve KNN performance, irrelevant features are usually removed through feature selection or dimensionality reduction (such as PCA), which increases preprocessing costs and model complexity.

Performance with Imbalanced Data

Majority class bias

In datasets with imbalanced classes, KNN **tends to predict new samples as majority classes** because the majority class samples have a higher proportion in the nearest neighbors, resulting in **low minority class recognition rates (such as missed diagnosis in disease diagnosis)**.

Boundary sample misjudgment

Minority class samples may be surrounded by majority class samples, resulting in their true neighbors being ignored, further exacerbating classification bias and affecting the fairness and practicality of the model.

Need to combine sampling techniques

To alleviate the imbalance problem, additional oversampling (such as **SMOTE**) or undersampling methods need to be used, but this may introduce overfitting or information loss, increasing the difficulty of tuning.

What is SMOTE?

SMOTE (Synthetic Minority Over-sampling Technique) balances imbalanced datasets by **creating artificial examples** of the minority class (e.g., rare diseases in medical data). It works by:

Step 1: For each minority-class sample, find its k nearest neighbors (using Euclidean distance).

Step 2: Generate new synthetic samples along the line between the original sample and a randomly chosen neighbor.

Formula:

$$NewSample = x_i + \alpha \cdot \left(x_{neighbor} - x_i\right)$$

where α is a random value between 0 and 1.

How KNN Uses SMOTE

- Two Critical Roles of KNN:
  - 1.In SMOTE:
    - KNN identifies the nearest neighbors of minority samples to create synthetic data.
    - Parameter: k_smote (number of neighbors used for sampling).
  - 2.As a Classifier:
    - After balancing data with SMOTE, KNN predicts labels by majority vote among k_classifier nearest neighbors.

- Important: k_smote (for SMOTE) and k_classifier (for KNN) are separate parameters needing independent tuning.

KNN with SMOTE : Advantages and Limitations

- **Advantages:**
  - **Boosts Minority-Class Detection**: Ideal for fraud detection or rare disease diagnosis.
- **Improves Metrics:**
  - ↑ Recall (reduces false negatives).
  - ↑ AUC-ROC (e.g., from 82% → 85% in sentiment analysis cases).
  - Handles Small Datasets: Effectively enlarges minority samples.

- **Limitations:**
  - **Noisy Samples:** SMOTE can amplify outliers if the minority class has noise.
  - **High k_smote:** Large values may create unrealistic synthetic data.
  - **Categorical Data:** Requires variants like SMOTEN (standard SMOTE works only for numerical features).

05 Practical Applications

Image Recognition

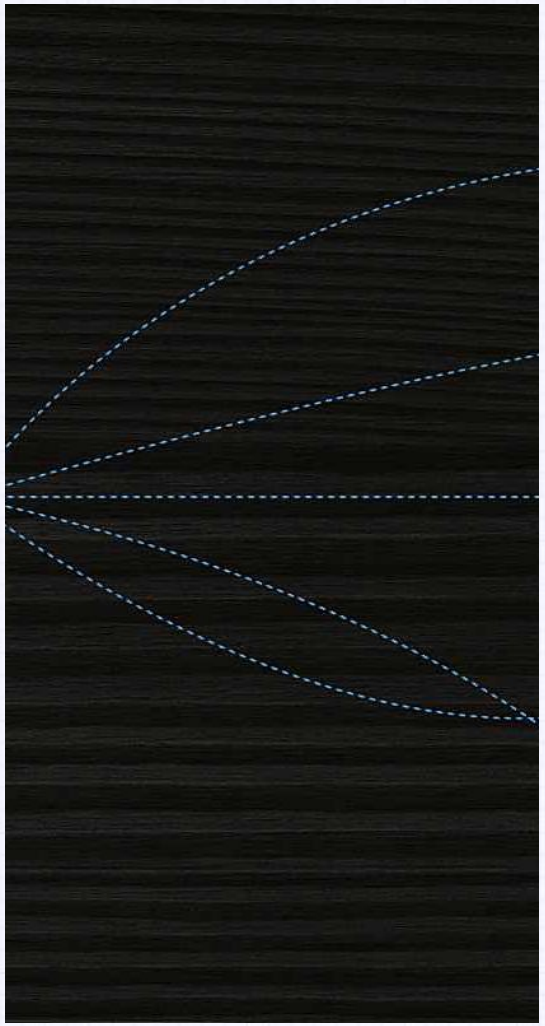
- Handwritten digit recognition: The KNN algorithm performs well on the MNIST dataset, and can quickly achieve high-precision classification by calculating the pixel distance between the number to be recognized and the training sample. It is commonly used in postal systems and bank bill processing.

- Facial feature matching: Combined with tools such as OpenCV, KNN can achieve facial recognition based on keypoint distance comparison, which is suitable for security systems and mobile device unlocking scenarios.

- Medical imaging analysis: In X-ray or CT image classification, KNN uses texture feature distance to determine lesion type, assisting doctors in early screening for pneumonia, tumors, and other conditions.

- Satellite image classification: Using KNN to cluster remote sensing image pixels can efficiently identify land cover types (such as forests/water bodies/cities), serving environmental monitoring and urban planning.

Recommendation Systems



Collaborative filtering recommendation

KNN calculates user behavior vector distance (such as rating matrix) to achieve "similar user like content" recommendation for e-commerce platforms, significantly improving conversion rates.

Content similarity matching

In the news recommendation scenario, KNN recommends articles with semantic features similar to the user's historical reading based on TF-IDF or Word2Vec vectorized text.

Real time personalized recommendation

Combined with a streaming computing framework, KNN can dynamically update the set of neighbors, enabling video platforms to implement drama recommendations based on users' real-time viewing behavior.

Medical Diagnosis

Disease risk prediction

KNN can predict the probability of chronic diseases such as diabetes and heart disease by analyzing the distance between clinical indicators (such as blood pressure/blood sugar) and historical cases.

Gene expression classification

In bioinformatics, KNN processes microarray gene data and achieves cancer subtype classification based on expression profile similarity, with an accuracy of over 85%.

Drug response assessment

Using KNN to compare patient genomes with drug trial databases, predict personalized medication efficacy, and reduce the incidence of adverse reactions.

Epidemiological analysis

Based on the calculation of symptom spatial distance, KNN can quickly identify suspected clustered cases of infectious diseases and assist in public health decision-making.

06 Improving KNN Performance

Feature Scaling

01

Z-score Normalization

Convert feature data into a distribution with a mean of 0 and a standard deviation of 1, using the formula (z= frac {x - mu} { sigma}), suitable for scenarios with large differences in feature scales and following a normal distribution, such as mixed age and income features.

02

Normalization processing (Min Max Scaling)

Linearly map features to the [0,1] interval, with the formula (x '= frac {x - text {min} { text {max} - text {min}}), suitable for non negative data such as image pixels or text word frequency, to avoid large numerical features dominating distance calculation.

03

Robust Scaling

Using median and quartile scaling (IQR), insensitive to outliers, suitable for datasets with noise or outliers, such as extreme values in medical testing.

Dimensionality Reduction

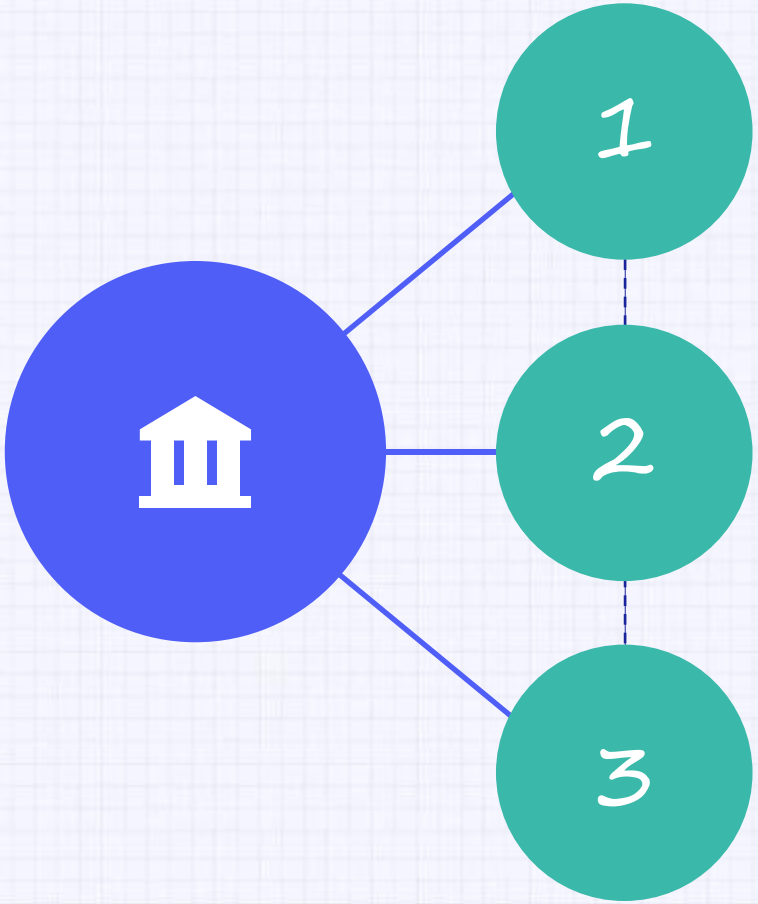
- Principal Component Analysis (PCA): By projecting high-dimensional data onto a low dimensional orthogonal space through linear transformation, the maximum variance direction is preserved, effectively reducing computational complexity and alleviating the "curse of dimensionality", but interpretability may be lost.

- T-SNE (t-Distributed Stochastic Neighbor Embedding): A nonlinear dimensionality reduction method that excels in visualizing the local structure of high-dimensional data and is suitable for exploratory data analysis. However, it has high computational complexity and the results are greatly affected by hyperparameters.

- Feature selection (Filter/rapper methods): Based on statistical tests (such as chi square tests)
- or model feedback (such as recursive feature elimination), key features are selected to directly eliminate redundant or irrelevant features, improving model efficiency.

- LDA (Linear Discriminant Analysis): Supervised dimensionality reduction technique that maximizes the ratio of inter class dispersion to intra class dispersion, suitable for classification tasks, but assuming that the data follows a Gaussian distribution and the covariance of each class is the same.

Optimizing K with Cross-Validation



Grid Search

The system traverses the preset candidate range of K values (such as 1-20), combines k-fold cross validation to select the best K, balances bias and variance, but the computational cost increases exponentially with the number of parameter combinations.

The Elbow Method

Draw error rate curves corresponding to different values of K, select the "inflection point" where the error decreases and slows down as K, which is intuitive but relies on subjective judgment and is suitable for preliminary rapid screening.

Leave One Out CV

an **extreme form of cross validation** where only **one sample is left for testing at a time**. It is suitable for small datasets and can fully utilize the data, but the computational cost is high, which may overestimate the model's generalization ability.

01 Introduction to Clustering

Definition and Purpose

- Data grouping technique: Clustering is an unsupervised learning method aimed at dividing objects in a dataset into several groups (clusters), such that objects within the same cluster have higher similarity, while objects between different clusters have lower similarity. Its core purpose is to discover the intrinsic structure and patterns in the data.

- Similarity measurement: Clustering relies on distance or similarity measures (such as Euclidean distance, cosine similarity) to divide clusters by calculating the proximity between data points, which is suitable for complex relationship analysis in multidimensional data spaces.

- Exploratory analysis tools: Clustering is commonly used for data preprocessing, anomaly detection, or feature extraction, helping researchers understand data distribution without relying on labels and providing a basis for subsequent decision-making.

Key Applications

- Computer vision: In image segmentation, clustering algorithms such as K-means can group pixels by color or texture to achieve object recognition or background separation, and are widely used in medical image analysis and autonomous driving scenarios.

- Market segmentation: Enterprises use cluster analysis of customer purchasing behavior or demographics to divide target customer groups, develop personalized marketing strategies, and improve conversion rates and user retention rates.

- Bioinformatics: Clustering gene expression data can identify functionally similar gene clusters, assisting in disease classification or drug target discovery, such as playing a key role in cancer subtype research.

- Recommendation system: Based on user historical behavior clustering (such as collaborative filtering), the platform can recommend content with similar user preferences, enhancing the personalized experience of services such as Netflix or Amazon.

Types of Clustering

Partition clustering: represented by K-means, the data is divided into K mutually exclusive clusters through iterative optimization, suitable for spherical distribution data, but sensitive to the initial center point and requiring pre specification of the number of clusters.

Hierarchical clustering: divided into cohesive (bottom-up merging) and split (top-down segmentation), generating a tree structure (dendrogram), suitable for revealing multi-scale hierarchical relationships in data, but with high computational complexity.

Density clustering: such as DBSCAN, forms clusters of arbitrary shapes based on data density and can automatically identify noise points. It is suitable for non-uniform distribution data, but is sensitive to parameter settings such as neighborhood radius and minimum number of points.

02 Clustering Algorithms

K-Means Clustering

- Efficiency and Scalability: K-Means has become one of the most widely used clustering algorithms due to its high computational efficiency and ease of implementation, especially suitable for processing large-scale datasets, and can further optimize performance through parallelization.

- Clear objective function: The algorithm achieves clustering by minimizing the squared error within the cluster (SSE), which facilitates theoretical analysis and parameter tuning.

- It is necessary to preset the number of clusters K: this is its main limitation, and it requires the assistance of elbow rule or contour coefficient to determine the optimal K value, otherwise it may lead to suboptimal clustering results.

K-Means Formula

K-Means aims to minimize the **Within-Cluster Sum of Squares (WCSS)**. The objective function is:

$$J = \sum_{i=1}^{N} \sum_{k=1}^{K} z_{i,k} x_i - \mu_k \quad ^2$$

Where:

x_i: i-th data point (dimension D).

μ_k: Centroid of cluster k.

$z_{i,k} \in \{0,1\}$: Binary indicator (1 if x_i belongs to cluster k, else 0).

$\cdot$: Euclidean distance:

$$d(a, b) = \sqrt{\sum_{j=1}^{D} \left(a_j - b_j\right)^2}$$

K-Means Algorithm Steps

K-Means iteratively optimizes J:

1. **Initialize**: Randomly select K initial centroids (e.g., choose points A1, A4, A7).
2. **Assign Points**: Assign each point to the nearest centroid's cluster:

$$\mathcal{C}_k = \{i: k = \text{argmin}_k x_i - \mu_k \quad ^2\}$$

3. **Update Centroids**: Recalculate centroids as the mean of assigned points:

$$\mu_k = \frac{1}{|\mathcal{C}_k|} \sum_{i \in \mathcal{C}_k} x_i$$

4. **Check Convergence**: Stop if centroids stabilize or max iterations reached; else repeat Steps 2–5.

K-Means Examples

Example 1: 1D Data (Simplified)

Data: {1, 2, 5, 6, 7}, $K = 2$ clusters.

Initial centroids: $\mu_1 = 5$, $\mu_2 = 6$.

Iteration 1:

Assign:

Cluster 1 (near $\mu_1 = 5$): {1, 2, 5} → New $\mu_1 = (1 + 2 + 5)/3 = 2.67$

Cluster 2 (near $\mu_2 = 6$): {6, 7} → New $\mu_2 = (6 + 7)/2 = 6.5$

Update: Centroids = $(2.67, 6.5)$.

Iteration 2:

Assign:

Cluster 1 (near *2.67*): {1, 2} → New $\mu_1 = (1 + 2)/2 = 1.5$

Cluster 2 (near *6.5*): {5, 6, 7} → New $\mu_2 = (5 + 6 + 7)/3 = 6$

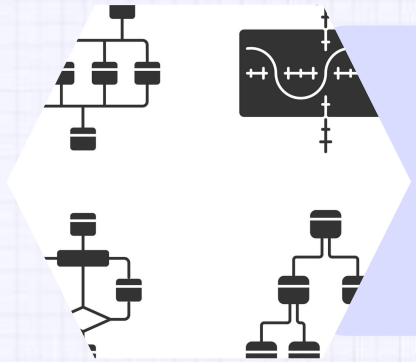
Update: Centroids = $(1.5, 6)$.

Iteration 3: No changes → **Converged**.

WCSS (J):

$$J = (1 - 1.5)^2 + (2 - 1.5)^2 + (5 - 6)^2 + (6 - 6)^2 + (7 - 6)^2 = 0.25 + 0.25 + 1 + 0 + 1 = 2.5$$

Hierarchical Clustering

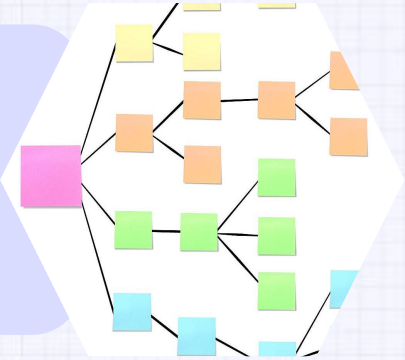


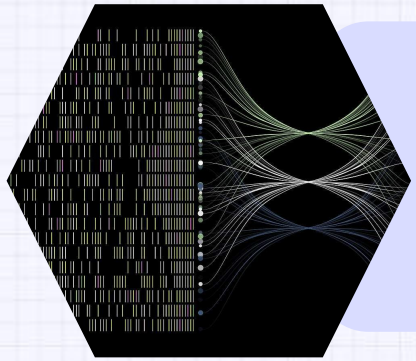
No need to preset the number of clusters

clustering results can be obtained at any level by cutting the tree diagram, which is highly flexible and suitable for exploratory data analysis.

Multi scale analysis capability

capable of simultaneously observing macro and micro cluster structures, such as for multi-level classification of gene expression data in bioinformatics.





High computational complexity

The time complexity is usually O (n^3), which is not suitable for large-scale datasets and needs to be combined with sampling or optimization algorithms (such as BIRCH) to improve efficiency.

Hierarchical Clustering - Core Formulas

a) Euclidean Distance (measures similarity between points):

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

where x, y are data point vectors.

b) Linkage Criteria (define cluster proximity):

Single Linkage: Minimum distance between clusters

$$d(C_a, C_b) = \min_{x \in C_a, y \in C_b} d(x, y)$$

Complete Linkage: Maximum distance between clusters

$$d(C_a, C_b) = \max_{x \in C_a, y \in C_b} d(x, y)$$

Lance-Williams Algorithm (general update formula):

$$d\left(C_i \cup C_j, C_k\right) = \alpha_i d(C_i, C_k) + \alpha_j d\left(C_j, C_k\right) + \beta d\left(C_i, C_j\right) + \gamma \left| d(C_i, C_k) - d\left(C_j, C_k\right) \right|$$

For single linkage: $\alpha_i = \alpha_j = \frac{1}{2}, \beta = 0, \gamma = -\frac{1}{2}$

Hierarchical Clustering

- **Algorithm Workflow:**
  - Starts with each point as a singleton cluster.
  - Iteratively merges the two closest clusters.
  - Updates distances using linkage criteria after each merge.
  - Ends with one all-inclusive cluster.
- **Parameter Impact:**
  - Distance Metric: Euclidean (used here) vs. Manhattan vs. Cosine.
  - Linkage Method: Single linkage (chosen) vs. complete linkage vs. average linkage.
  - Results vary significantly with these choices.

Hierarchical Clustering

- **Complexity:**
  - Requires memory for distance matrix.
  - Suitable for datasets with points.
- **Implementation:**
  - Use Python's scipy.cluster.hierarchy.linkage with parameters:
  - from scipy.cluster.hierarchy import linkage
  - Z = linkage(points, method='single', metric='euclidean')
- **Common Misconceptions:**
  - Differs fundamentally from centroid-based methods (e.g., K-means) and density-based methods (e.g., DBSCAN).
  - Generates nested clusters (dendrogram), not flat partitions.

DBSCAN

- Automatic recognition of cluster quantity: Discovering clusters of any shape through neighborhood radius (ε) and minimum number of points (minPts) parameters, without the need for preset K values, adept at handling noise points and outliers.

- Adapt to complex distributions: Robust to complex structures such as non spherical clusters and nested clusters, such as urban clustering detection in geographic spatial data.

- The selection of ε and minPts directly affects the results and requires parameter tuning through methods such as k-distance graphs. High dimensional data may lose density definition due to the "curse of dimensionality".

- The performance of datasets with uneven density is poor, and improved variants such as OPTICS algorithm need to be combined to solve the problem.

DBSCAN Core Formulas & Definitions

Term	Formula/Definition
ε-neighborhood	$N_\varepsilon(p) = q \in D \mid dist(p,q) \leq \varepsilon$
Core Point	$N_\varepsilon(p) \geq MinPts$
Directly Density-Reachable	Point q is directly reachable from p if: p is core point and $q \in N_\varepsilon(p)$
Cluster	A maximal set of density-connected points
Noise	Points not in any cluster (labeled -1)

DBSCAN Example

Dataset (6 points from):

Cluster 1: A(1, 1), B(1, 2), C(2, 1)
Cluster 2: D(6, 6), E(6, 7), F(7, 6)

Parameters:

$\varepsilon = 1.5$
$MinPts = 3$

Step-by-Step Calculation:

1. **Check A(1,1)**:

- $dist(A,B) = \sqrt{(1-1)^2 + (2-1)^2} = 1.0 \leq 1.5 \rightarrow$ **include**
- $dist(A,C) = \sqrt{(2-1)^2 + (1-1)^2} = 1.0 \leq 1.5 \rightarrow$ **include**
- $dist(A,others) > 4.0 \rightarrow$ **exclude**
- $N_\varepsilon(A) = B, C \rightarrow N_\varepsilon = 2 < 3 \rightarrow$ **Not core**

2. **Check B(1,2)**:

- $N_\varepsilon(B) = A, C$ (distances: $A = 1.0, C = \sqrt{2} \approx 1.41 \leq 1.5$)
- $N_\varepsilon(B) = 2 < 3 \rightarrow$ **Not core**

3. **Check C(2,1)**:

- $N_\varepsilon(C) = A, B \rightarrow N_\varepsilon = 2 < 3 \rightarrow$ **Not core**

4. **Points D, E, F**:

- All have $N_\varepsilon = 2$ (e.g., $N_\varepsilon(D) = E, F$) $\rightarrow$ **No core points**

Result:

No clusters → All points labeled **noise (-1)**
Why? $N_\varepsilon = 2 < MinPts(3)$ for all points.

Fix Parameters (as in):

Set $\varepsilon = 1.6$ *or* $MinPts = 2$:
B and E become **core points** ($N_\varepsilon \geq 2$).
Clusters form: {A,B,C} and {D,E,F}.

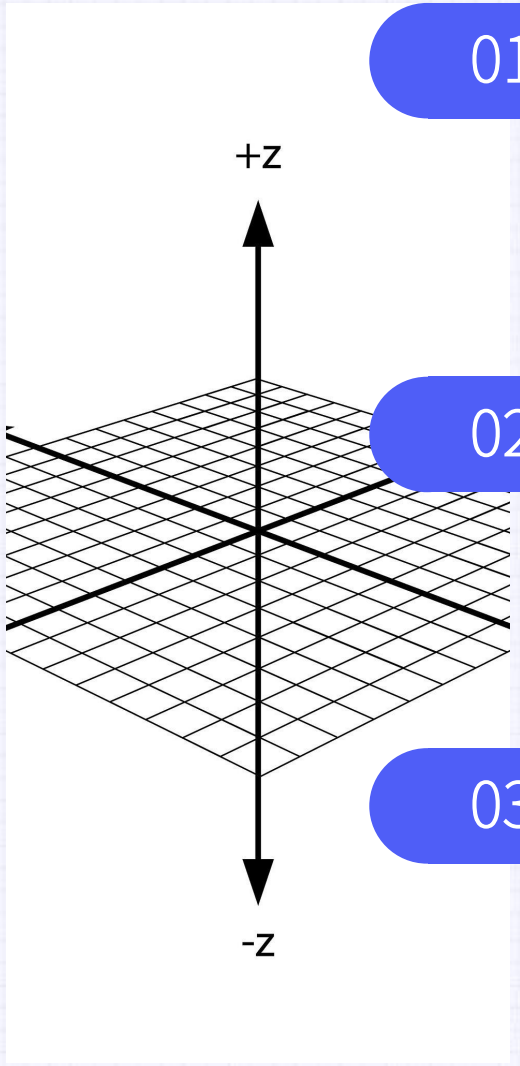
DBSCAN Parameter Selection Tips

- 1.ε: Use a k-distance plot (find "elbow" where distance spikes).
- 2.MinPts: Start with (e.g., 3 for 2D data).
- 3.Debugging:
- All noise? → Decrease ε or decrease MinPts.
- One giant cluster? → Increase ε or increase MinPts.

- DBSCAN excels at finding irregular shapes () but fails with uniform density clusters (as shown above). Always validate parameters visually or with metrics like silhouette score.

03 Distance Metrics in Clustering

Euclidean Distance



01

Geometric space measurement

Euclidean distance is the most common distance measurement method, which calculates the straight-line distance between two points in multidimensional space and is suitable for continuous data clustering. The formula is $\sqrt{}(\Sigma\ (x_i-y_i)^2)$.

02

Scale sensitivity

As the Euclidean distance is directly affected by the size of the eigenvalues, it needs to be standardized (such as Z-score normalization) to avoid bias caused by dimensional differences.

03

Spherical cluster applicability

It performs well when the data distribution is spherical or approximately spherical, but has poor performance on high-dimensional sparse data or non convex shaped clusters (susceptible to the "curse of dimensionality").

Manhattan Distance

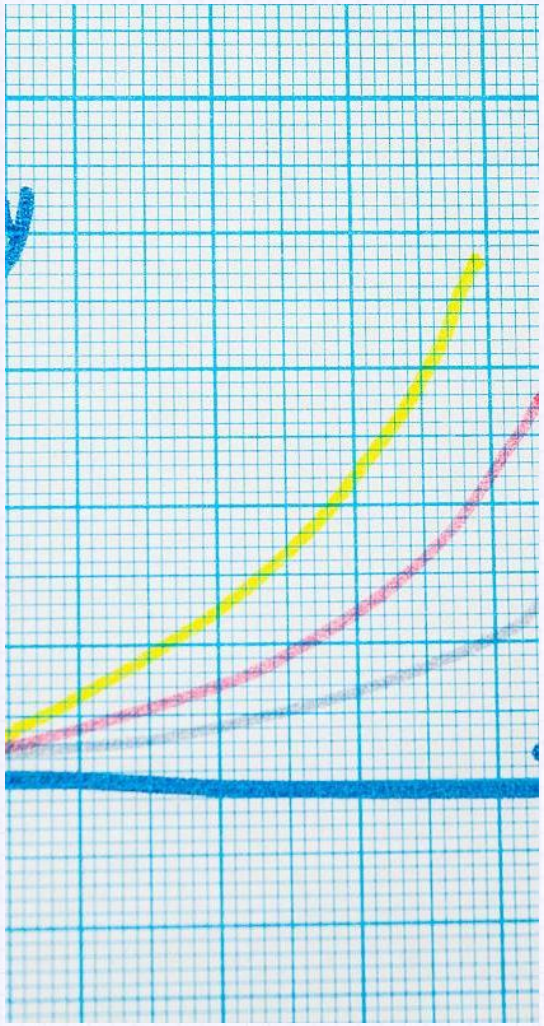
- Urban block measurement: also known as L1 distance, calculates the sum of absolute differences in each dimension ($\Sigma \mid x_i - y_i \mid$), simulates the walking distance of grid paths, and is suitable for datasets with discrete features.

- Robustness advantage: Compared to Euclidean distance, it is less sensitive to outliers and more stable in the presence of noisy data or outliers. It is commonly used in scenarios such as taxi path planning.

- Coordinate axis alignment characteristics: The effect is significant when the data features have a clear axial distribution, but are sensitive to rotational changes and require principal component analysis (PCA) preprocessing.

- Sparse data processing: Especially suitable for processing high-dimensional sparse data (such as text TF-IDF vectors), it can effectively capture the differences between features rather than amplitude differences.

Cosine Similarity



Direction similarity measurement

Evaluate sample direction similarity by calculating the cosine value of the vector angle (A · B)/(||A|| · ||B||), which is insensitive to vector length and suitable for high-dimensional data such as text and images.

Advantages of document retrieval

Outstanding in the field of information retrieval, it can effectively handle the similarity comparison of document word frequency vectors and solve the problem of "word frequency differences but semantic similarity".

Normalization property

The output range is fixed at [-1,1] and can be directly used as a similarity index. It is often combined with TF-IDF/PCA to improve clustering performance, but is sensitive to feature weight allocation.

04 Evaluation of Clustering Results

Silhouette Score

- The core indicator for measuring clustering quality: silhouette coefficient quantifies the tightness of samples within clusters and the separation between clusters, providing an intuitive method for evaluating clustering performance, especially suitable for unsupervised learning scenarios lacking real labels.
- Strong interpretability: Its value range is between [-1, 1]. The closer the value is to 1, the better the clustering effect. If it is close to 0 or negative, it suggests the possibility of overlapping clusters or misclassification, providing clear direction for model optimization.
- Wide applicability: Supports multiple measurement methods such as Euclidean distance and cosine distance, and can be flexibly applied to different algorithms such as K-means and hierarchical clustering. It is a universal tool for cross domain clustering analysis.

Silhouette Score of Clustering

- The **Silhouette Score measures clustering quality** by quantifying how well samples are grouped within their own cluster (cohesion) versus how separated they are from other clusters (separation).

For a single sample *i*:

$$s(i) = \frac{b(i) - a(i)}{\max\left(a(i), b(i)\right)}$$

Where:

$a(i)$: Average distance between sample *i* and **all other samples in the same cluster**.

(Measures cohesion; smaller = better).

$b(i)$: Average distance between sample *i* and **all samples in the nearest neighboring cluster** (the cluster with the smallest average distance).

(Measures separation from the next-best cluster; larger = better).

Overall Score (for all samples):

$$S = \frac{1}{N} \sum_{i=1}^{N} s(i)$$

(Average of all individual $s(i)$ scores, where N = total samples).

Interpretation

1: Perfect clustering (tight clusters, well-separated).

0: Overlapping clusters (sample is on the decision boundary).

−1: Poor clustering (sample assigned to wrong cluster).

Davies-Bouldin Index

High computational efficiency

Only intra cluster dispersion and inter cluster center distance are needed for fast calculation, suitable for efficient evaluation of large-scale datasets.

Strong robustness

insensitive to noisy data and outliers, able to stably reflect the overall quality of clustering structure.

Complementary to silhouette coefficient

Focusing on the global evaluation of inter cluster separation, it can be combined with silhouette coefficient to comprehensively analyze clustering performance.

Davies-Bouldin Index of Clustering

- The Davies-Bouldin Index (DBI) is an internal validation metric for evaluating clustering quality.
- It measures the balance between intra-cluster compactness and inter-cluster separation.
- A lower DBI indicates better clustering (tight clusters that are well-separated).

The standard DBI formula is:

$$DBI = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i}\left(\frac{s_i + s_j}{d(c_i, c_j)}\right)$$

Symbols:

k: Number of clusters.

s_i: **Average intra-cluster distance** for cluster i. Calculated as the mean Euclidean distance between each point in cluster i and its centroid:

$$s_i = \frac{1}{|C_i|}\sum_{x \in C_i} \|x - c_i\|_2$$

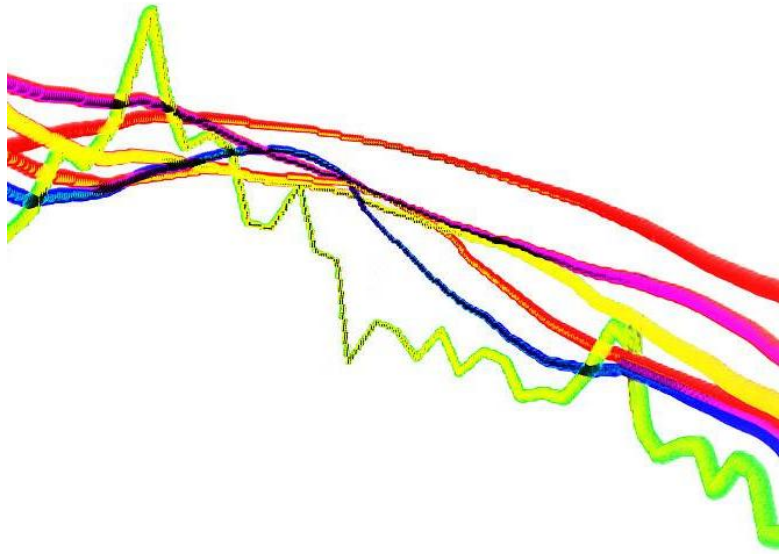
where $|C_i|$ is the number of points in cluster i, and c_i is the centroid of cluster i.

$d(c_i, c_j)$: **Euclidean distance between centroids** c_i and c_j:

$$d(c_i, c_j) = \|c_i - c_j\|_2$$

$\max_{j \neq i}$: For cluster i, find the cluster j ($j \neq i$) that maximizes the ratio $\frac{s_i + s_j}{d(c_i, c_j)}$. This identifies the "most similar" (least separated) cluster to i.

Elbow Method



By plotting the sum of squared errors (SSE) curves corresponding to different numbers of clusters (K values), the optimal K value is determined by observing the "inflection point". The core assumption is that the reasonable number of clusters is reached when the rate of SSE decline significantly slows down.

Elbow Method

It needs to be combined with manual judgment

the identification of turning points may be subjective and usually requires verification with indicators such as contour coefficients.

Suitable for distance based algorithms such as K-means

Significant performance on data with spherical cluster distribution, but may fail on non convex clusters or data with uneven density.

Balancing computational costs

As the K value increases, the model needs to be trained repeatedly, which may consume more resources in high-dimensional data.

Elbow Method of Clustering

- The Elbow Method evaluates clustering quality using Within-Cluster Sum of Squares (WCSS) (also called Sum of Squared Errors (SSE) or Inertia). The formula is:

$$\text{WCSS/SSE} = \sum_{i=1}^{k} \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

k : Number of clusters

C_i : Points in cluster i

x_j : A data point in C_i

μ_i : Centroid of C_i

$\|x_j - \mu_i\|^2$: Squared Euclidean distance between point x_j and centroid μ_i

Key Insight: As k increases, SSE decreases. The "elbow point" (optimal k) occurs where SSE's rate of decrease sharply slows.

05 Challenges in Clustering

Determining Optimal Clusters

Model selection criticality

The number of clusters directly affects algorithm performance, and too many or too few can lead to overfitting or underfitting, which needs to be scientifically evaluated through methods such as elbow rule and contour coefficient.

Domain knowledge dependency

The optimal number of clusters in real-world scenarios often needs to be determined based on business logic, such as customer segmentation standards, which may deviate from actual needs by relying solely on mathematical indicators.

Handling High-Dimensional Data

Dimension curse impact

Euclidean distance fails in high-dimensional space, resulting in distortion of similarity measurement, and alternative solutions such as Mahalanobis distance or cosine similarity need to be adopted.

Feature selection strategy

Reduce dimensionality through principal component analysis (PCA) or t-SNE, retain over 90% of the variance in principal components, and visualize intermediate results to validate effectiveness.

Adaptive optimization of algorithms

Select dimension insensitive methods such as spectral clustering or DBSCAN to avoid performance degradation of k-means in high-dimensional space.

Sensitivity to Initial Conditions

The k-means++algorithm improves convergence stability by 15-20% compared to random initialization by probabilistically selecting the initial centroid.

The strategy of multiple restarts (usually 50-100 times) can reduce the risk of local optima and, when combined with parallel computing, reduce time overhead.

Non convex distribution data requires density clustering (such as OPTICS) to automatically identify cluster structures through reachable distance maps.

Isolation mechanisms need to be set up for noise data processing, such as optimizing the core point determination radius ε and minimum neighborhood point minPts parameters in DBSCAN.

06 Advanced Topics in Clustering

Density-Based Clustering

- The ability to handle clusters of arbitrary shapes: Traditional distance based clustering methods (such as K-means) are only suitable for spherical cluster distributions, while density clustering can effectively identify clusters with complex topological structures, such as circular, spiral, and other non convex datasets, by analyzing the local density distribution of data points.

- Automatic noise filtering function: By setting a density threshold, data points in sparse areas can be marked as noise or outliers, significantly improving the robustness of clustering results in noisy environments, suitable for the common problem of noise interference in real scenes.

- No need to preset the number of clusters: Unlike algorithms that require pre specified K values, density clustering (such as DBSCAN) dynamically generates clusters based on the characteristics of the data itself, avoiding the influence of subjective parameter selection on the results.

Spectral Clustering

01

The application of Turalapras matrix: using similarity matrix to construct graph structure, by performing feature decomposition on Laplacian matrix, projecting data into feature space, transforming complex nonlinear relationships in the original space into linearly separable form.

02

No strong assumptions about sample distribution: does not rely on data following a specific distribution (such as Gaussian distribution), performs well in manifold learning scenarios, such as recognizing groups with similar patterns but far apart in images or social networks.

03

Parameter sensitivity analysis: Careful selection of similarity measurement methods (such as Gaussian kernel bandwidth) is necessary. Improper parameters may lead to over segmentation or under segmentation. Parameter optimization is usually combined with eigenvalue gap method or heuristic strategy.

Spectral Clustering

Spectral clustering projects data into a spectral space where clusters become separable, making it ideal for complex structures like spirals or concentric rings

Algorithm Steps :

a) Affinity Matrix (Similarity Graph)

Defines pairwise similarities using a Gaussian kernel (RBF):

$$W_{ij} = \exp\left(-\frac{x_i - x_j^2}{2\sigma^2}\right)$$

σ: Bandwidth parameter controlling neighborhood size (e.g., $\sigma = 0.2$ or 1).

Alternative: Binary similarity $W_{ij} = 1$ if $x_i - x_j < \Theta$, else 0 .

b) Degree Matrix

Diagonal matrix where:

$$D_{ii} = \sum_{j=1}^{n} W_{ij}$$

$D_{ij} = 0$ for $i \neq j$.

c) Laplacian Matrix

Normalized symmetric form (recommended):

$$L = I - D^{-1/2} W D^{-1/2}$$

I: Identity matrix.

Unnormalized form: $L = D - W$.

d) Eigen Decomposition & Clustering

1. Compute the first k eigenvectors $v_1, , v_k$ of L (smallest eigenvalues).
2. Stack eigenvectors into matrix $U \in \mathbb{R}^{n \times k}$ (rows = data points in spectral space).
3. Apply **k-means** to rows of U to assign clusters (e.g., $M = 2$ or 3).

Clustering with Deep Learning

- Extracting low dimensional representations of high-dimensional data through autoencoders or variational autoencoders (VAEs), eliminating redundant features and preserving clustering key information, and improving the performance of subsequent traditional clustering algorithms such as K-means.

- Joint optimization frameworks, such as Deep Embedded Clustering (DEC), directly optimize clustering objectives while learning feature representations. By iteratively updating clustering centers and feature distributions, end-to-end unsupervised learning is achieved.

- Cluster models based on Generative Adversarial Networks (GANs), such as ClusterGAN, separate generated samples from different clusters through hidden space and optimize inter cluster separation using discriminator feedback. They are suitable for high-dimensional complex data such as images.

- Diffusion models learn data distribution through a gradual denoising process and combine latent variables to partition cluster boundaries, demonstrating potential in text and cross modal data.